ADAPTIVE GOAL-ORIENTED AI OPERATING LAYER WITH AUTONOMOUS WORKFLOW LEARNING

A PROJECT REPORT

Submitted by

HARINI.D. S **(2116231901009)**

KEERTHIKA.S **(2116231901024)**

in partial fulfillment for the award of the degree

of

BACHELOR OF ENGINEERING

in

COMPUTER SCIENCE AND ENGINEERING (CYBER SECURITY)





RAJALAKSHMI ENGINEERING COLLEGE

ANNA UNIVERSITY: CHENNAI 600 025

OCTOBER 2026

RAJALAKSHMI ENGINEERING COLLEGE
CHENNAI

BONAFIDE CERTIFICATE

Certified that this Project report titled "**ADAPTIVE GOAL-ORIENTED AI OPERATING LAYER WITH AUTONOMOUS WORKFLOW LEARNING**" is the bonafide work of "**HARINI D S (2116231901009), KEERTHIKA S (2116231901024)**" who carried out the work under my supervision. Certified further that to the best of my knowledge the work reported herein does not form part of any other project report or dissertation on the basis of which a degree or award was conferred on an earlier occasion on this or any other candidate.



SIGNATURE
Dr. P. Tamilselvi M.E., Ph.D.,
HEAD OF THE DEPARTMENT

PROFESSOR

Department of Computer Science and Engineering (Cyber Security)

Rajalakshmi Engineering College
Chennai – 602 105



SIGNATURE
Ms. Gayathripriya P S
SUPERVISOR

ASSISTANT PROFESSOR

Department of Computer Science and Engineering (Cyber Security)

Rajalakshmi Engineering College
Chennai – 602 105

Submitted to Project Viva-Voce Examination held on 06-10-2026



Internal Examiner



External Examiner

ABSTRACT

Traditional operating systems execute explicit commands, while users generally think in terms of goals such as organizing files, opening applications, or researching information. The Adaptive Goal-Oriented AI Operating Layer is a Windows desktop assistant designed to bridge this gap. It provides a compact conversational interface that converts natural-language goals into structured workflow plans, requests the capabilities required for those plans, obtains user approval, and executes only allowlisted actions through specialized agents. The system is implemented as a React and TypeScript frontend connected to a FastAPI backend. Its workflow engine combines goal interpretation, task planning, capability-scoped permissions, real automation for files, applications, and public web navigation, persistent SQLite memory, workflow learning, and failure recovery. A WebSocket event bus provides live execution updates to the interface. Security is treated as a primary design concern: arbitrary shell or code execution is excluded, destructive operations require confirmation, and path validation restricts file activity to approved scopes. The system demonstrates how an AI operating layer can make desktop automation more understandable, permission-aware, personalized, and recoverable. The repository includes unit tests for the principal backend components and end-to-end scripts for running the service-dependent workflows. Current limitations include the need for target-machine dependency verification, a recovery engine focused on common path failures, public-web browser patterns, and a tray integration that can be enhanced into a fully native floating window.

ACKNOWLEDGEMENT

First, we thank the almighty God for the successful completion of the project. Our sincere thanks to our Chairman **Mr. S. Meganathan B.E., F.I.E.,** for his sincere endeavor in educating us in his premier institution. We would like to express our deep gratitude to our beloved Chairperson **Dr. Thangam Meganathan Ph.D.,** for her enthusiastic motivation which inspired us a lot in completing this project and Vice Chairman **Mr. Abhay Shankar Meganathan B.E., M.S.,** for providing us with the requisite infrastructure.

We also express our sincere gratitude to our college Principal **Dr. S.N. Murugesan M.E., Ph.D.,** and **Dr. P. Tamilselvi M.E., Ph.D., Professor and Head, Department of Computer Science and Engineering (Cyber Security)** and our project guide **Ms. Gayathri Priya** for her encouragement and guiding us throughout the project. We would like to thank our project coordinator **Mr. Benedict J.N., M.E., Assistant Professor (SG),** for his encouragement and guidance towards successful completion of this project and to our parents, friends, all faculty members and supporting staffs for their direct and indirect involvement in successful completion of the project for their encouragement and support.

We also acknowledge the open-source communities behind FastAPI, React, Vite, SQLAlchemy, Playwright, SQLite, pytest, and the other tools used in this project.

HARINI D S - 2116231901009
KEERTHIKA S - 2116231901024

TABLE OF CONTENTS

CHAPTER NO.	TITLE	PAGE NO.
	ABSTRACT	**iii**
	LIST OF TABLES	**vi**
	LIST OF FIGURES	**vii**
1	**INTRODUCTION**	**1**
	1.1 BACKGROUND	1
	1.2 PROBLEM STATEMENT	1
	1.3 OBJECTIVES	1
	1.4 SCOPE	2
	1.5 SECURITY REQUIREMENTS	3
	1.6 SDG ALIGNMENT	3
2	**LITERATURE REVIEW**	**4**
	2.1 EXISTING SYSTEM WITH SECURITY APPROACHES	4
	2.2 RESEARCH GAPS	4
	2.3 PROPOSED SYSTEM	4
	2.3.1 OVERVIEW OF THE PROPOSED SYSTEM	4
	2.3.2 OBJECTIVES OF THE PROPOSED SYSTEM	5
	2.3.3 ADVANTAGES OF PROPOSED SYSTEM	5
3	**SYSTEM DESIGN**	**7**
	3.1 SYSTEM ARCHITECTURE	7
	3.2 MODULE DESIGN	9
	3.3 UML DIAGRAMS	10
	3.4 THREAT MODEL AND ATTACK SURFACE	13
	3.5 SECURITY CONTROLS	13
4	**SYSTEM DESCRIPTION**	**15**
	4.1 DATA COLLECTION	15
	4.2 DATA PREPROCESSING	16
	4.2.1 ALGORITHMS	16
	4.2.2 RESULTS AND DISCUSSION	17
5	**CONCLUSION**	**19**
	REFERENCES	**21**
	APPENDIX	**22**

LIST OF TABLES

TABLE NO.	TITLE	PAGE NO.
2.1	COMPARISON OF TRADITIONAL DESKTOP TOOLS AND THE PROPOSED SYSTEM	5
3.1	API SURFACE	10
3.2	SECURITY CONTROLS	13

LIST OF FIGURES

FIGURE	TITLE	PAGE NO.
3.1	ARCHITECTURE OVERVIEW	8
3.2	USE CASE DIAGRAM	11
3.3	WORKFLOW LIFECYCLE	12
3.4	GOAL-TO-EXECUTION SEQUENCE	13
3.5	SECURITY VALIDATION PIPELINE	14
4.1	PERSISTENT DATA MODEL	15

CHAPTER 1
INTRODUCTION

1.1 BACKGROUND

Most operating systems expose low-level actions and application interfaces, but they do not natively maintain a durable understanding of what the user is trying to accomplish. Users therefore repeat sequences of file operations, application launches, browser searches, and verification steps. Existing assistants can interpret natural language and suggest actions, but they often separate language understanding from controlled execution, long-term memory, and user-visible approval. The system addresses this gap as an operating layer that sits above Windows. It does not replace the operating system. Instead, it coordinates a natural-language interface, a workflow engine, security boundaries, real automation adapters, and persistent memory.

Motivation

- Reduce manual effort in frequently repeated desktop workflows.
- Make automation transparent through plans, permissions, approvals, and status updates.
- Personalize assistance using preferences, execution history, and learned workflows.
- Improve reliability through verification, structured error reporting, and recovery.
- Provide a security-conscious alternative to unrestricted AI-generated shell or code execution.

1.2 PROBLEM STATEMENT

The project addresses the problem of repetitive desktop work being executed as isolated commands rather than as user goals. The system must interpret a goal, break it into ordered tasks, identify the capabilities required, obtain appropriate approval, perform the actions, verify outcomes, recover from common failures, and remember successful workflows without exposing the user to uncontrolled automation.

1.3 OBJECTIVES

- Develop an AI-powered operating layer for Windows desktop workflows.
- Interpret user goals expressed in natural language.
- Convert goals into ordered, executable, validated tasks.
- Coordinate multiple agents for files, applications, browser actions, memory, and recovery.
- Enforce least-privilege permissions and allowlisted actions.
- Maintain long-term preferences, workflow history, and execution statistics.
- Detect repeated successful workflows and offer them as learned automations.
- Provide reliable failure reporting, recovery for common path issues, and live status updates.

1.4 SCOPE

Scope of the Present Implementation

- File operations include create, find, list, move, copy, rename, combine, delete, open in Explorer, and verification actions.
- Application automation supports launching installed applications after validation.
- Browser automation supports public URL navigation, web search, and downloads through Playwright.
- The frontend provides onboarding, goal input, plan review, permission dialogs, approval, and workflow status.
- The backend exposes REST APIs for goals, workflows, memory, learning, permissions, and system context, plus a WebSocket workflow stream.
- The project targets Windows desktop usage and uses a local SQLite database for persistence.

Out of Scope

- The system is not a replacement operating system or a general unrestricted computer-control agent.

- The current version does not provide arbitrary shell execution from model output.
- The present browser layer is focused on public navigation and search patterns rather than authenticated web applications.
- The current recovery engine is not a general planner capable of resolving every possible execution failure.

1.5 SECURITY REQUIREMENTS

Non-Functional Requirements

- Security: reject actions not present in the allowlist and require confirmation for destructive actions.
- Usability: present understandable plans, permissions, and execution feedback before and during automation.
- Reliability: verify operations, record execution history, and preserve structured failure reasons.
- Maintainability: keep agents, API routes, automation adapters, memory, security, and learning as separate modules.
- Performance: use a local service and asynchronous FastAPI/WebSocket communication for responsive updates.
- Portability within target scope: support Windows-specific desktop integration while keeping core workflow logic modular.

1.6 SDG ALIGNMENT

S. No.	SDG No. & Goal	Relevant SDG Target	Project Contribution / Alignment
1	**SDG 9** Industry, Innovation and Infrastructure	**Target 9.5** Enhance scientific research and upgrade technological capabilities, encouraging innovation.	Introduces an AI-powered operating layer that turns natural-language goals into planned, permission-checked desktop workflows using FastAPI, React and multi-agent automation.

S. No.	SDG No. & Goal	Relevant SDG Target	Project Contribution / Alignment
2	**SDG 8** Decent Work and Economic Growth	**Target 8.2** Achieve higher levels of economic productivity through technological upgrading and innovation.	Reduces manual effort in repeated file, application and browser tasks, and learns repeated successful workflows for reuse.
3	**SDG 16** Peace, Justice and Strong Institutions	**Target 16.6** Develop effective, accountable and transparent institutions at all levels.	Makes automation transparent and accountable through visible plans, scoped permissions, user approval, allowlisted actions and structured execution history.

CHAPTER 2
LITERATURE REVIEW

2.1 EXISTING SYSTEM WITH SECURITY APPROACHES

Common AI assistants such as Microsoft Copilot, Apple Intelligence, Open Interpreter, and AutoGPT can understand natural-language requests and support basic computer operations.

- **Natural-language assistance:** Often focused on a single request rather than a reusable workflow.
- **Desktop automation:** May depend on predefined operations or broad permissions.
- **AI suggestions:** Does not necessarily verify the final state of the computer.
- **Personalization:** User preferences and history may not persist in a structured local store.
- **Failure handling:** Recovery may require manual intervention or reissuing the request.

2.2 RESEARCH GAPS

However, the project identifies several gaps relevant to reliable desktop automation: limited workflow learning, inconsistent long-term preference storage, unclear authorization boundaries, weak recovery semantics, and limited integration between multiple specialized agents.

2.3 PROPOSED SYSTEM

2.3.1 OVERVIEW OF THE PROPOSED SYSTEM

The system provides a controlled pipeline: the user expresses a goal, the interpreter produces structured tasks, the planner validates task order and capabilities, the permission manager checks requested scopes, the user approves the plan, and the executor routes each action to a specialized agent. Results are verified and stored in SQLite. Repeated successful workflows are evaluated by the learning engine and can be saved as learned workflows after user decision.

A conventional AI assistant stops at a text response and leaves execution, verification, and memory to the user, whereas the proposed system carries a goal through planning, permission, approval, scoped execution, verification, and learning.

2.3.2 OBJECTIVES OF THE PROPOSED SYSTEM

Contributions

- A goal-first workflow pipeline from natural-language intent to executable tasks.
- A multi-agent design with dedicated goal, planning, file, browser, application, memory, and recovery agents.
- Capability and scope-aware permission enforcement with an allowlisted action registry.
- SQLite-backed persistence for goals, workflows, tasks, execution history, permissions, and preferences.
- A learning engine that identifies repeated, successful workflows for user-approved reuse.
- A React floating assistant with onboarding, approval, permission, and execution-status experiences.

2.3.3 ADVANTAGES OF PROPOSED SYSTEM

Comparative View

Table 2.1 Comparison of Traditional Desktop Tools and the Proposed System

FEATURE	TRADITIONAL DESKTOP TOOLS	Proposed System
Interaction model	Commands and menus	Natural-language goals
Planning	Manual or predefined	Structured task plans
Authorization	Application-level or implicit	Capability and scope aware
Execution	Separate tools	Specialized agents through one executor
Memory	Limited or fragmented	SQLite preferences and history
Learning	Usually external scripting	Success-threshold workflow candidates
Recovery	Manual retry	Structured recovery engine and verification

Feasibility

- **Technical:** Feasible: the implementation uses mature Python, FastAPI, React, SQLite, Playwright, and pytest components.
- **Operational:** Feasible for a local Windows desktop deployment with user approvals and configured dependencies.
- **Economic:** Suitable for a student project because the core stack is open source; an OpenAI API key is optional for model-backed interpretation.
- **Security:** Feasible through allowlisted actions, scope checks, confirmation flags, path validation, and structured logging.
- **Schedule:** Modular components allow incremental delivery and testing of the core workflow path.

CHAPTER 3
SYSTEM DESIGN

3.1 SYSTEM ARCHITECTURE

The system is organized into presentation, API, workflow-engine, agent/automation, security, and persistence layers. A goal moves through the system as follows: natural-language input -> interpretation -> planning -> capability and scope checks -> user approval -> execution -> verification and recovery -> persistence and learning.

- **Presentation:** Main components: React, TypeScript, Vite, Tailwind, floating assistant Responsibility: Onboarding, goal entry, plan display, permission and approval interaction, live status.
- **API:** Main components: FastAPI routes, Uvicorn, WebSocket Responsibility: Expose goal, workflow, memory, learning, permissions, and context operations.
- **Workflow engine:** Main components: GoalInterpreter, TaskPlanner, AutomationExecutor Responsibility: Interpret, validate, route, execute, and coordinate workflows.
- **Agents:** Main components: Goal, Planner, File, Browser, Application, Memory, Recovery Responsibility: Perform specialized reasoning, automation, storage, and recovery responsibilities.
- **Security:** Main components: ActionRegistry, PermissionManager, Validator, PathValidator Responsibility: Allowlist actions, enforce capabilities/scopes, confirm destructive work, validate paths.
- **Persistence:** Main components: SQLite, SQLAlchemy, MemoryManager, LearningEngine Responsibility: Store goals, workflows, tasks, execution history, permissions, preferences, and learned workflows.

Figure 3.1 presents the layered architecture of the system. The presentation layer communicates with the FastAPI layer over REST and WebSocket; the workflow engine coordinates the specialized agents; the security layer is consulted before any action is planned or executed; and the persistence layer stores goals, workflows,

history, and learned workflows. The agents act on the Windows desktop environment only through allowlisted, permission-checked operations.

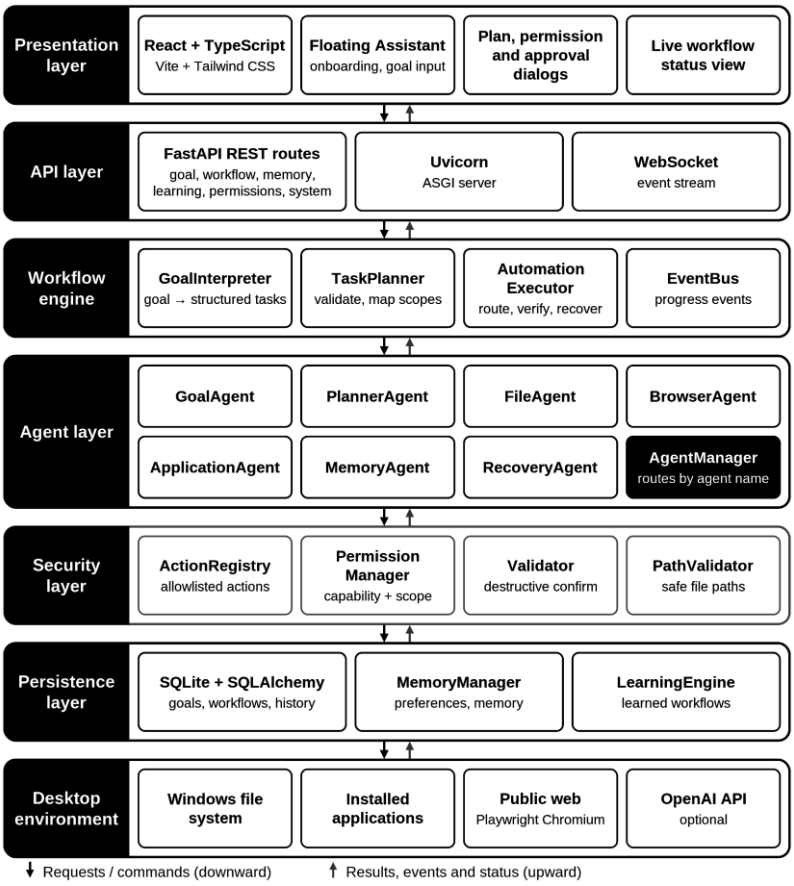


Figure 3.1 Architecture Overview

Technology Stack

- **Backend:** Python 3.12+, FastAPI, Uvicorn, Pydantic
- **AI integration:** OpenAI provider abstraction and structured goal interpreter
- **Persistence:** SQLite, SQLAlchemy, python-dotenv
- **Automation:** pathlib, shutil, psutil, Playwright, pywin32, pystray, Pillow
- **Frontend:** React, TypeScript, Vite, Tailwind CSS
- **Testing:** pytest, HTTPX, Playwright-oriented workflow scripts
- **Desktop integration:** Windows tray launcher and compact assistant interaction

Backend Services

The FastAPI application creates a shared Services object at startup. This object initializes the database, action registry, memory manager, permission manager,

recovery engine, event bus, interpreter, planner, agent manager, executor, learning engine, and context engine. API routers are separated by concern, which keeps the public surface understandable and allows each feature to evolve independently.

3.2 MODULE DESIGN

Functional Requirements

- **FR1:** Requirement: Accept a natural-language user goal. Implementation evidence: GoalInterpreter and goal API.
- **FR2:** Requirement: Generate an ordered workflow plan. Implementation evidence: TaskPlanner, PlannerAgent, and WorkflowPlan model.
- **FR3:** Requirement: Request and enforce permissions. Implementation evidence: PermissionManager, permission routes, and capability scopes.
- **FR4:** Requirement: Execute controlled real-world actions. Implementation evidence: AutomationExecutor and specialized agents.
- **FR5:** Requirement: Persist results and preferences. Implementation evidence: SQLAlchemy models and SQLite database.
- **FR6:** Requirement: Learn repeated successful workflows. Implementation evidence: LearningEngine threshold and learned-workflow records.
- **FR7:** Requirement: Report progress and errors. Implementation evidence: EventBus and WebSocket endpoint.
- **FR8:** Requirement: Recover from common failures. Implementation evidence: RecoveryEngine and RecoveryAgent.

Multi-Agent Design

- **GoalAgent:** Uses the goal interpreter to convert intent into structured tasks.
- **PlannerAgent:** Creates a validated workflow plan and identifies required capabilities.
- **FileAgent:** Performs controlled file-system operations after permission and path checks.
- **BrowserAgent:** Handles public URL navigation, search, and browser downloads.
- **ApplicationAgent:** Launches validated installed applications.

- **MemoryAgent:** Reads and updates preferences, history, and workflow records.
- **RecoveryAgent:** Applies recovery logic to common execution failures.
- **AgentManager:** Registers agents and provides controlled routing by agent name.

The AutomationExecutor delegates work through the AgentManager to seven specialized agents. The agents are grouped by responsibility, and every action-performing agent is subject to the security layer before it reaches the file system, browser, or installed applications.

Frontend Experience

The frontend starts by checking persisted memory for a user name. If no preference is found, it presents onboarding; otherwise, it displays the floating assistant. The interface is intentionally compact and task-oriented: it supports goal entry, plan explanation, permission dialogs, approval, progress, and completion feedback rather than functioning as a generic chat window.

API Surface

Table 3.1 API Surface

ENDPOINT	PURPOSE
GET /api/health	Service health check.
POST /api/goal/interpret	Interpret a natural-language goal.
POST /api/workflow/plan	Create a structured plan.
POST /api/workflow/approve	Record user approval.
POST /api/workflow/execute	Execute an approved workflow.
GET /api/workflow/history	Retrieve workflow history.
GET /api/memory	Retrieve preferences and memory.
POST /api/memory/preferences	Save user preferences.
GET /api/learning/workflows	List learned workflows and candidates.
POST /api/permissions/request	Request capability permission.
POST /api/permissions/grant	Grant a capability for a scope.
POST /api/permissions/revoke	Revoke a permission.
GET /api/system/context	Read local system context.

ENDPOINT	PURPOSE
WS /ws/workflow/{workflow_id}	Stream live workflow events.

3.3 UML DIAGRAMS

Figure 3.2 presents the use cases supported by the system for its primary actor, the user. The included and extended use cases in the second column are performed by the system as part of executing a workflow.

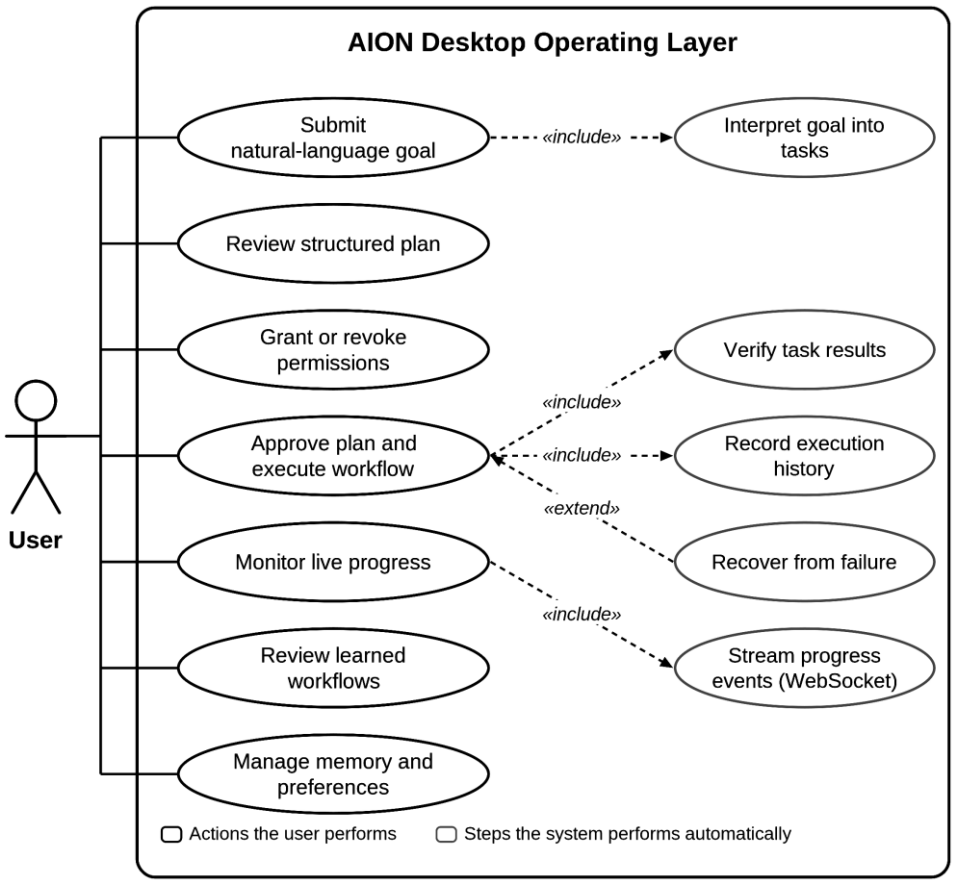


Figure 3.2 Use Case Diagram

Figure 3.3 summarizes the workflow lifecycle as a flowchart. It shows the two approval gates (permissions and plan approval), the live status channel provided by the EventBus, the verification step after execution, and the recovery path that retries or replans when a task does not complete.

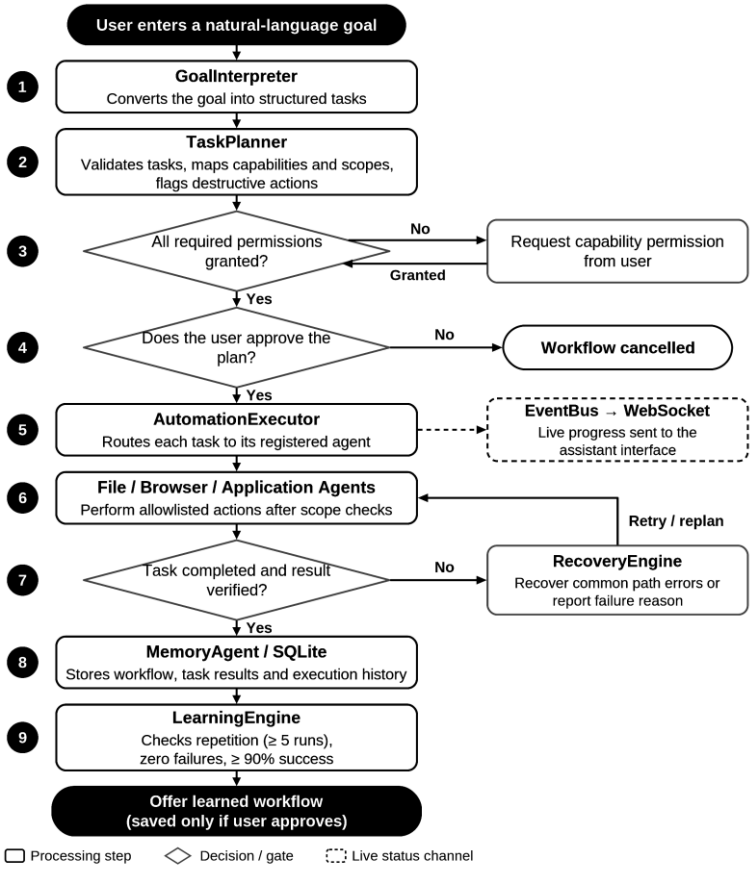


Figure 3.3 Workflow Lifecycle

Figure 3.4 shows the order of interactions between the interface, the API routes, the workflow components, and the database when a goal is interpreted, planned, approved, and executed. Dashed arrows represent responses and live events.

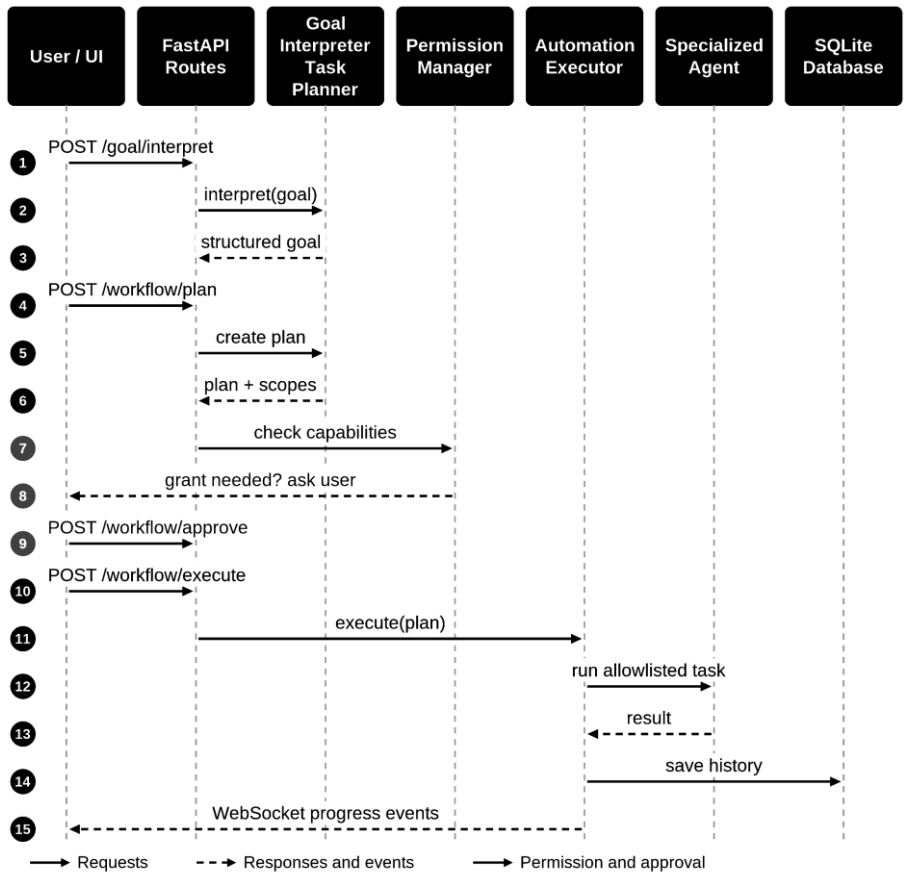


Figure 3.4 Goal-to-Execution Sequence

On startup the interface checks persisted memory for a user name and either presents onboarding or the floating assistant, which then guides the user through goal entry, plan explanation, permissions, approval, progress, and completion feedback.

3.4 THREAT MODEL AND ATTACK SURFACE

- An AI-generated task could attempt an action outside the intended capability.
- A destructive file action could cause irreversible data loss.
- A path parameter could target a protected or unauthorized directory.
- A browser action could navigate to an unintended destination.
- Sensitive values could be exposed through logs or persisted state.
- A failed task could leave the computer in a partially changed state.

3.5 SECURITY CONTROLS

Table 3.2 Security Controls

CONTROL	HOW THE SYSTEM APPLIES IT
Allowlisted actions	Only actions registered in AllowedActionRegistry can be planned and executed.
Capability checks	Actions map to file_access, browser_access, or application_access capabilities.
Scope checks	Permissions are associated with a capability and an explicit directory or scope.
Destructive confirmation	Delete actions are marked as requiring confirmation by the planner.
Path validation	File automation validates allowed locations and protects against unsafe paths.
No arbitrary execution	The system does not execute arbitrary shell commands or model-generated code.
Structured logging	Execution messages and errors are stored as structured records rather than opaque transcripts.

Figure 3.5 illustrates the sequence of security gates that every planned action must pass. A failure at any gate blocks the action, returns the reason to the user, and is recorded in structured logs.

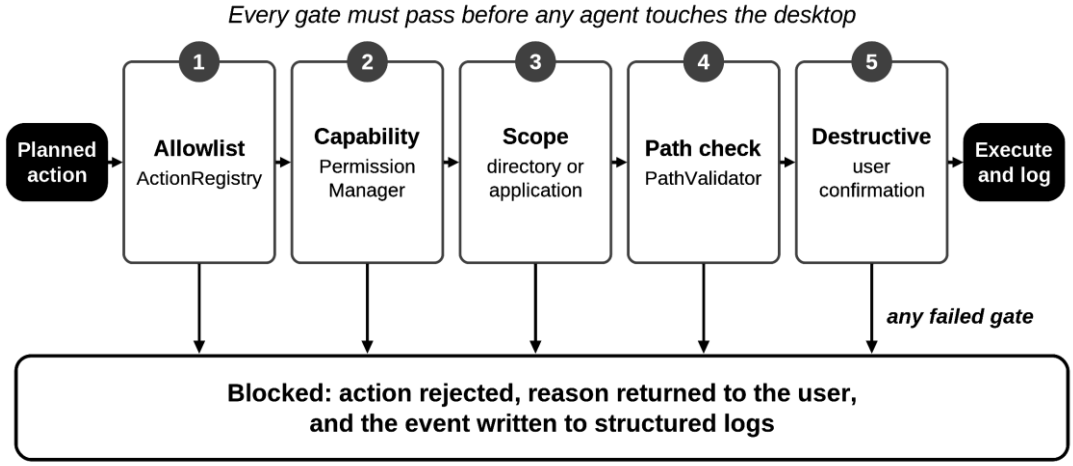


Figure 3.5 Security Validation Pipeline

CHAPTER 4
SYSTEM DESCRIPTION

4.1 DATA COLLECTION

Data Model

The persistence model contains GoalRecord, WorkflowRecord, WorkflowTaskRecord, ExecutionHistoryRecord, PermissionRecord, and UserPreferenceRecord. Workflow records retain execution_count, success_count, failure_count, average execution time, approval state, learned state, and timestamps. This enables both auditability and learning decisions without requiring raw conversations to be treated as the only source of state.

Figure 4.1 illustrates the persistent data model used by the system. A goal produces workflows, each workflow contains ordered tasks and accumulates execution history, while permissions and user preferences are stored as user-level state in the local SQLite database.

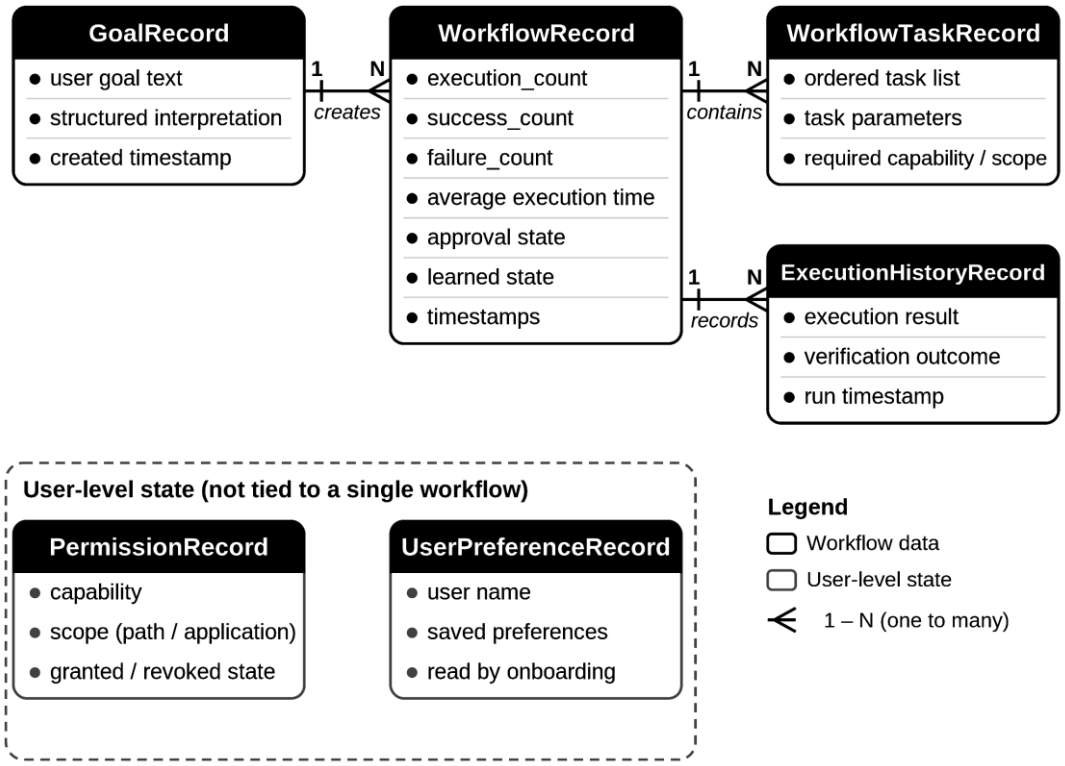


Figure 4.1 Persistent Data Model

Privacy and Local Storage

The system stores preferences, workflow metadata, task parameters, execution history, and permissions in a local SQLite database. This supports personalization while keeping the primary state on the user's machine. The project still requires careful deployment hygiene: environment files must not be committed, API credentials must be protected, and sensitive paths or task parameters should be reviewed before sharing logs or database files.

4.2 DATA PREPROCESSING

Workflow Lifecycle

- The user enters a goal in the compact assistant interface.
- GoalInterpreter converts the natural language request into a structured task representation.
- TaskPlanner validates task models, resolves action rules, records required capabilities and scopes, and marks destructive tasks for confirmation.
- The frontend presents the plan and requests any missing permissions.
- The user approves the workflow; the backend records the approved state.
- AutomationExecutor routes each task to the agent registered for the action.
- The executor publishes progress events, handles recoverable failures, verifies results where required, and records execution history.
- LearningEngine evaluates repetition and success rate and offers qualifying workflows for user-approved learning.

4.2.1 ALGORITHMS

Workflow Learning

The current LearningEngine uses a conservative threshold of five executions. A workflow is considered a candidate only when it has reached the threshold, has no recorded failures, has a success rate of at least 0.90, and has not already been marked learned. Saving a candidate is an explicit user decision, preserving user control over automation that becomes reusable.

A workflow progresses from tracked executions to a learned workflow. Promotion is conservative and always requires an explicit user decision.

Failure Recovery

RecoveryEngine handles common recoverable file-path scenarios and returns structured recovery payloads. Non-recoverable failures remain visible with explicit reasons. This design separates error interpretation from action execution and creates a foundation for richer retry and replanning strategies.

The RecoveryEngine interprets the error separately from execution; recoverable file-path scenarios return a structured recovery payload, while non-recoverable failures are reported with an explicit reason.

4.2.2 RESULTS AND DISCUSSION

Test Strategy

The repository contains focused pytest modules for the action registry, agent manager, application agent, automation executor, browser agent, file agent, goal interpreter, learning engine, memory manager, permission manager, recovery engine, and task planner. This coverage aligns tests with the project's main ownership boundaries rather than treating the whole system as one opaque unit.

- **Action registry:** Confirm allowed actions, capabilities, assigned agents, and destructive flags.
- **Goal interpreter:** Validate conversion of goals into structured task representations.
- **Task planner:** Confirm task validation, capability aggregation, and confirmation marking.
- **Permission manager:** Check grant, revoke, and scope-aware permission behavior.
- **File and application agents:** Exercise controlled local automation behavior.
- **Memory and learning:** Validate persistence, history statistics, learning thresholds, and learned workflows.
- **Recovery engine:** Check structured recovery responses for recoverable and non-recoverable cases.
- **Executor:** Validate routing and execution coordination.

The testing strategy has three levels: isolated unit tests for each major component, HTTP-based integration scripts that require a running backend, and end-to-end demonstration workflows.

Validation Observation

A full pytest invocation from the repository root currently attempts to collect several root-level debug and end-to-end scripts. Some of those scripts make direct HTTP requests to 127.0.0.1:8000 during collection, so they fail when the FastAPI backend is not already running. This is an environment-dependent integration-test issue rather than evidence that the isolated unit test modules are invalid. For reproducible validation, the focused tests under the tests directory should be run independently, while service-dependent scripts should be run only after starting the backend and configuring the environment.

```
python -m pytest tests -q
python -m uvicorn backend.main:app --reload --host
127.0.0.1 --port 8000
```

Demonstration Workflows

- PDF organization: create an Demo folder in Downloads, find PDF files, move them, verify the result, and persist the execution.
- Browser research: open a public search workflow through BrowserAgent and retain execution history.
- Learning detection: repeat a successful workflow until the learning threshold is met, then approve saving it as a learned workflow.

Results and Discussion

The implementation provides a coherent end-to-end design for controlled goal-oriented desktop automation. Its strongest results are architectural: responsibilities are separated, security checks are part of planning and execution, workflow state is persisted, and the interface exposes approval and progress. The main remaining validation work is deployment-level execution on the target Windows machine with dependencies, Playwright Chromium, environment variables, and a running backend configured correctly.

CHAPTER 5
CONCLUSION

The system presents an adaptive AI operating layer that treats desktop automation as a controlled workflow rather than an isolated command. The system accepts a user goal, interprets it, plans an ordered set of tasks, checks capabilities and scopes, obtains approval, routes the work to specialized agents, publishes progress, verifies outcomes, stores history, and learns from repeated success. Its architecture combines a React assistant interface, FastAPI services, SQLite persistence, real file/application/browser automation, a multi-agent manager, security validation, and failure recovery.

The project's central contribution is the integration of intelligence and control. The system aims to make automation useful without making it opaque: users can see what will happen, authorize the capabilities required, observe execution, and decide whether repeated behavior should become a learned workflow. With further runtime validation and the enhancements identified in this report, the project can evolve into a more capable and secure personalized computing layer.

Current Limitations

- Target-machine dependency installation and runtime verification are still required for a complete deployment test.
- Browser automation currently focuses on public navigation and search patterns.
- Recovery is implemented for common path issues and can be expanded to richer replanning.
- The tray integration opens the compact UI in a browser; a fully native always-on-top floating window remains future work.
- The learning engine uses a fixed threshold and success-rate rule rather than a richer policy or recommendation model.
- The system depends on carefully configured permissions and user approval; usability may need refinement for complex multi-scope workflows.

Future Enhancements

- Introduce richer NLU with schema-correction loops and stronger multi-step plan validation.
- Implement a native always-on-top assistant window anchored to the tray icon.
- Add voice input and output for hands-free goal entry.
- Extend the policy engine with fine-grained risk levels and per-action confirmation policies.
- Add workflow optimization recommendations with explainable user approval.
- Provide richer history, memory, and agent-activity views in the frontend.
- Expand recovery with alternative plans, safe retries, rollback support, and state-aware verification.
- Add stronger privacy controls such as encrypted local storage and configurable retention for history.

REFERENCES

1. S. Yao, J. Zhao, D. Yu, N. Du, I. Shafran, K. Narasimhan, and Y. Cao, "ReAct: Synergizing reasoning and acting in language models," in *Proc. 11th Int. Conf. Learn. Represent. (ICLR)*, 2023. [Online]. Available: https://arxiv.org/abs/2210.03629

2. T. Schick, J. Dwivedi-Yu, R. Dessì, R. Raileanu, M. Lomeli, L. Zettlemoyer, N. Cancedda, and T. Scialom, "Toolformer: Language models can teach themselves to use tools," in *Adv. Neural Inf. Process. Syst. (NeurIPS)*, vol. 36, 2023. [Online]. Available: https://arxiv.org/abs/2302.04761

3. Y. Shen, K. Song, X. Tan, D. Li, W. Lu, and Y. Zhuang, "HuggingGPT: Solving AI tasks with ChatGPT and its friends in Hugging Face," in *Adv. Neural Inf. Process. Syst. (NeurIPS)*, vol. 36, 2023. [Online]. Available: https://arxiv.org/abs/2303.17580

4. L. Wang et al., "A survey on large language model based autonomous agents," *Front. Comput. Sci.*, vol. 18, no. 6, Art. no. 186345, 2024, doi: 10.1007/s11704-024-40231-1. [Online]. Available: https://arxiv.org/abs/2308.11432

5. Q. Wu et al., "AutoGen: Enabling next-gen LLM applications via multi-agent conversation," *arXiv preprint arXiv:2308.08155*, 2023. [Online]. Available: https://arxiv.org/abs/2308.08155

6. T. Xie et al., "OSWorld: Benchmarking multimodal agents for open-ended tasks in real computer environments," in *Adv. Neural Inf. Process. Syst. (NeurIPS)*, vol. 37, 2024. [Online]. Available: https://arxiv.org/abs/2404.07972

7. S. Zhou et al., "WebArena: A realistic web environment for building autonomous agents," in *Proc. 12th Int. Conf. Learn. Represent. (ICLR)*, 2024. [Online]. Available: https://arxiv.org/abs/2307.13854

8. K. Greshake, S. Abdelnabi, S. Mishra, C. Endres, T. Holz, and M. Fritz, "Not what you've signed up for: Compromising real-world LLM-integrated applications with indirect prompt injection," in *Proc. 16th ACM Workshop Artif. Intell. Secur. (AISec)*, 2023, pp. 79–90, doi: 10.1145/3605764.3623985. [Online]. Available: https://arxiv.org/abs/2302.12173

9. J. H. Saltzer and M. D. Schroeder, "The protection of information in computer systems," *Proc. IEEE*, vol. 63, no. 9, pp. 1278–1308, Sep. 1975, doi: 10.1109/PROC.1975.9939.

10. OWASP Foundation, "OWASP Top 10 for Large Language Model Applications," 2025. [Online]. Available: https://owasp.org/www-project-top-10-for-large-language-model-applications/

11. S. Ramírez, "FastAPI documentation." [Online]. Available: https://fastapi.tiangolo.com/

12. Meta Platforms, Inc., "React documentation." [Online]. Available: https://react.dev/

13. SQLAlchemy Authors, "SQLAlchemy documentation." [Online]. Available: https://docs.sqlalchemy.org/

14. Microsoft Corporation, "Playwright for Python documentation." [Online]. Available: https://playwright.dev/python/

15. SQLite Consortium, "SQLite documentation." [Online]. Available: https://www.sqlite.org/docs.html

APPENDIX

A PROJECT STRUCTURE

- **backend/agents:** Goal, planner, file, browser, application, memory, and recovery agents.
- **backend/api:** FastAPI route modules.
- **backend/automation:** Event bus, executor, file operations, application/browser adapters, and path validation.
- **backend/context:** Local system context services.
- **backend/learning:** Workflow learning engine.
- **backend/llm:** Provider abstraction and goal interpretation.
- **backend/memory:** Database initialization, models, and memory manager.
- **backend/planner:** Task planning and workflow plan generation.
- **backend/recovery:** Failure recovery engine.
- **backend/security:** Action registry, permissions, and validators.
- **frontend/src:** React application, components, pages, hooks, API service, and styles.
- **tests:** Focused unit tests for core backend modules.
- **database/aion.db:** SQLite persistence file used by the local application.

B INSTALLATION AND EXECUTION SUMMARY

- Install backend dependencies with python -m pip install -r backend/requirements.txt.
- Copy .env.example to .env and configure the OpenAI key and local service settings as needed.
- Install frontend dependencies by running npm install from the frontend directory.
- Install the Playwright Chromium browser with python -m playwright install chromium.
- Start the backend with python -m uvicorn backend.main:app --reload --host 127.0.0.1 --port 8000.

- Start the frontend with npm run dev from the frontend directory.
- Launch the tray integration with python backend/tray_app.py after the backend and frontend are available.

C IMPLEMENTATION STATUS SUMMARY

- **Natural-language goal interpretation:** Implemented through GoalInterpreter/provider abstraction.
- **Task planning and capability mapping:** Implemented through TaskPlanner and action registry.
- **Multi-agent orchestration:** Implemented through AgentManager and AutomationExecutor.
- **File/application/browser automation:** Implemented with scoped controls and adapters.
- **Persistent memory:** Implemented with SQLite and SQLAlchemy records.
- **Workflow learning:** Implemented with threshold-based candidate detection.
- **Failure recovery:** Implemented for common recoverable path scenarios.
- **Frontend assistant:** Implemented with onboarding and workflow interaction components.
- **Native desktop experience:** Partially implemented through tray integration; native floating window is future scope.
- **Full deployment validation:** Requires configured dependencies, running backend, and target-machine verification.